

第2章_数据信息的表示_知识点整理

第2章 数据信息的表示 — 知识点整理

2.1 数据表示的作用

2.1.1 数据表示的综合考虑因素

在设计和选择计算机内的数据表示方式时，一般需要综合考虑以下几方面因素：

因素	说明
数据的类型	数值数据（小数、整数、实数/浮点数等）、非数值数据（ASCII码、汉字等）
表示的范围和精度	通过选择适当的数据类型和字长来实现
存储和处理的代价	数据格式易于表示、存储和处理
软件的可移植性	数据格式符合相应的规范，方便软件在不同计算机之间的移植

2.2 数值数据的表示

2.2.1 真值与机器数

概念	说明
真值	带符号的二进制数，如+1011、-1100、+0.1011、-0.1100
机器数（机器码）	计算机中实际存储的编码形式，包括原码、反码、补码、移码

示例：

- 真值 +1011（十进制：+11）
 - 真值 -1100（十进制：-12）
 - 真值 +0.1011（十进制：+0.6875）
 - 真值 -0.1100（十进制：-0.75）
-

2.2.2 原码表示法

规则：正数的原码数值位为本身，符号位为0；负数的原码数值位为本身，符号位为1。

示例：

类型	真值	原码
正小数	$x=+0.1101$	$[x]_{\text{原}}=0.1101$
正整数	$x=+1101$	$[x]_{\text{原}}=0,1101$
负小数	$x=-0.1111$	$[x]_{\text{原}}=1.1111$
负整数	$x=-1111$	$[x]_{\text{原}}=1,1111$

0的原码：

类型	$[+0]_{\text{原}}$	$[-0]_{\text{原}}$
小数	0.0000	1.0000
整数	0,0000	1,0000

重要特性：+0的原码和-0的原码是不一样的。

2.2.3 反码表示法

规则：正数的反码数值位为本身，符号位为0；负数的反码数值位为本身取反，符号位为1。

示例：

类型	真值	反码
正小数	$x=+0.1101$	$[x]_{\text{反}}=0.1101$
正整数	$x=+1101$	$[x]_{\text{反}}=0,1101$
负小数	$x=-0.1111$	$[x]_{\text{反}}=1.0000$
负整数	$x=-1111$	$[x]_{\text{反}}=1,0000$

0的反码：

类型	$[+0]_{\text{反}}$	$[-0]_{\text{反}}$
小数	0.0000	1.1111

类型	[+0]反	[-0]反
整数	0,0000	1,1111

重要特性：+0的反码和-0的反码是不一样的。

关联：原码（知识点3）和反码（知识点4）都存在正负零的问题，补码（知识点5）解决了这个问题。

2.2.4 补码表示法 — 模的概念

核心思想：利用模运算将减法转换为加法。

时钟示例：模（模数）为12

- $15 \text{点} \equiv 12+3 \equiv 3 \pmod{12}$
- 11点调到3点：顺时针拨4小时或逆时针拨8小时
- $11+4 \equiv 3 \pmod{12}$, $11-8 \equiv 3 \pmod{12}$
- **结论：**-8与+4对模12是互补的，即-8的补码是+4

关键理解：

- 一个负数在某个模的前提下，可以找到其对应的一个正数（补码）
- 以12为模时：-2的补码是+10，-5的补码是+7

图片信息：PPT中有两个时钟图片（Picture6.jpg和Picture4.jpg），展示了顺时针和逆时针两种调法。

2.2.5 补码的数学定义

定点小数x的补码（形式为 $x_0.x_1x_2\cdots x_n$ ， x_0 为符号位，模为2）：

$$[x]_{\text{补}} = \begin{cases} x & 0 \leq x < 1 \\ 2 + x = 2 - |x| & -1 \leq x < 0 \end{cases}$$

定点整数x的补码（形式为 $x_0x_1x_2\cdots x_n$ ，共 $n+1$ 位，模为 2^{n+1} ）：

$$[x]_{\text{补}} = \begin{cases} x & 0 \leq x < 2^n \\ 2^{n+1} + x = 2^{n+1} - |x| & -2^n \leq x < 0 \end{cases}$$

图片信息：PPT中有两个公式图片（图片7.jpg和图片5.jpg），展示了定点小数和定点整数的补码公式。

2.2.6 求补码的示例

例2.1：求补码

真值x	补码 $[x]_{\text{补}}$	计算过程
+0.0101	0.0101	正数，数值位不变
-0.0101	1.1011	$2+x=2-0.0101=1.1011$
-0.0000	0.0000	$2+x=2-0.0000=2$ ，满模溢出本位归零
-1.0000	1.0000	$2+x=2-1.0000=1.0000$

例2.2（字长8位）：

- $x=(-10101)_2$: $[x]_{\text{补}}=1\ 0000\ 0000-10101=1110\ 1011$
- $x=-128$: $[x]_{\text{补}}=256-128=128=1000\ 0000$

例2.3：

- $x=-1000$: $[x]_{\text{补}}=1\ 0000-1000=1,1000$
- $x=-0001$: $[x]_{\text{补}}=1\ 0000-0001=1,1111$
- $x=-0.0001$: $[x]_{\text{补}}=2-0.0001=1.1111$

重要经验：某些机器码转换可依据经验公式（扫描法），但特殊形式需依据公式变换。

2.2.7 由补码求真值

例2.4：已知 $[x]_{\text{补}}=1,1000$ 和 $[x]_{\text{补}}=1,1111$ ，求真值

方法1：依据公式

- $x=1\ 0000-[x]_{\text{补}}=1\ 0000-1\ 1000=-1000$
- $x=1\ 0000-[x]_{\text{补}}=1\ 0000-1\ 1111=-0001$

方法2：扫描法

- 对真值数据位从右往左顺序扫描
- 遇到第一个1以及右侧的0保持不变
- 左侧全部取反
- 适合手动计算

方法3：反码法

- 真值数据位逐个取反，末位+1

- 适合硬件求补

2.2.8 补码求法总结

负数真值→补码：数值位取反加1

真值x	数值位	取反	加1	[x]补
-0.0101	0101	1010	1011	1.1011
-0.0000	0000	1111	0000	0.0000
-1.0000	0000	1111	0000	1.0000
$(-10101)_2$	10101	01010	01011	1110 1011
-128	1000 0000	0111 1111	1000 0000	1000 0000
-1000	1000	0111	1000	1,1000
-0001	0001	1110	1111	1,1111
-0.0001	0001	1110	1111	1.1111

负数补码→真值：数值位也是取反加1

0的补码（重要特性）：

- $[+0]_{\text{补}}=0.0000$, $[-0]_{\text{补}}=0.0000$
- +0的补码和-0的补码是一样的（与原码、反码不同）

关联：补码解决了原码和反码中正负零不唯一的问题，使得0的表示唯一，这是补码成为计算机中最常用有符号数表示法的关键原因。

2.2.9 变形补码（双符号补码，模4补码）

定义：使用两位符号位的补码，模为4。

例2.5：

- $x=-0.0101$: $[x]_{\text{补}}=4+x=4-0.0101=11.1011$
- $x=-10101$ (8位字长): $[x]_{\text{补}}=1110 1011$

溢出判断：

- 符号位为00或11 → 无溢出
- 符号位为01 → 正溢出

- 符号位为10 → 负溢出

示例（9位二进制整数，双符号位）：

运算	结果	双符号位	判断
27+45	72	00,100 1000	无溢出
27-45	-18	11,110 1110	无溢出
80+90	170	01,010 1010	正溢出
-80-90	-170	10,101 0110	负溢出

变形补码表示范围（9位，双符号位）：-128~+127（即11,000 0000~00,111 1111）

关键说明：变形补码双符号位中，最高位代表真实符号。

2.2.10 移码表示法

定义：移码为补码的符号位取反。

例2.6：

真值x	[x]补	[x]移
+1010110	0,1010110	1,1010110
-1010110	1,0101010	1,0101010

移码的符号位规则：

- 符号位为0时表示负数
- 符号位为1时表示正数
- 小数没有移码

0的移码：[+0]移=1,0000，[-0]移=1,0000（+0和-0的移码一样）

移码的两个主要用途：

用途	说明
比较阶码大小	用无符号数规则直接比较两个浮点数阶码的大小
判断"浮点机器0"	移码各位均为0时，对应阶码最小，若尾数也为全0，则为"浮点机器0"

比较示例（字长8位，-6与+5）：

- 补码比较: 1111 1010 vs 0000 0101 → 看成无符号数时1111 1010更大 (错误)
- 移码比较: 0111 1010 vs 1000 0101 → 1000 0101更大 (正确)

2.2.11 真值的补码和移码对照表

真值 (十进制)	二进制表示	补码 (8位)	移码 (8位)
-128	-1000 0000	1,000 0000	0,000 0000
-127	-0111 1111	1,000 0001	0,000 0001
...	...	...	...
-1	-0000 0001	1,111 1111	0,111 1111
0	0000 0000	0,000 0000	1,000 0000
1	0000 0001	0,000 0001	1,000 0001
...	...	...	...
127	0111 1111	0,111 1111	1,111 1111

关联: 移码与补码只有符号位不同 (知识点11), 这个对照表清晰展示了两者的关系。

2.2.12 4位不同机器码在数轴上的表示

4位二进制数整数:

机器码	表示范围
原码	-7~+7
反码	-7~+7
补码	-8~+7
移码	-8~+7

图片信息: PPT中有图2.2 (图片2.jpg), 展示了4位不同机器码在数轴上的表示。

关键观察: 补码和移码比原码、反码多表示一个负数 (-8), 因为补码/移码中0的表示唯一。

2.2.13 定点小数 (纯小数)

格式： $X = X_0.X_1X_2\cdots X_n$

部分	说明
X_0	符号位
$X_1 \sim X_n$	数值的有效部分（尾数）， x_1 为最高有效位

示例： $x = 0.1101$ （正数）， $x = 1.1001$ （负数）

2.2.14 定点整数（纯整数）

格式： $X = X_0X_1X_2\cdots X_n$

部分	说明
X_0	符号位
$X_1 \sim X_n$	数值的有效部分

示例： $x = 0,1101$ （正数）， $x = 1,1001$ （负数）

C语言中的定点整数：char（8位）、short（16位）、int（32位）、long（64位）

2.2.15 定点数表示范围

若计算机字长为 $n+1$ （含1位符号位），则可表示 2^{n+1} 个数据状态。

图片信息：PPT中有表2.4（图片7.jpg），展示了不同机器码的定点数表示范围。

数轴上的定点数表示：

- 定点整数：数轴上每个刻度间隔为1
- 定点小数：数轴上每个刻度间隔为 2^{-n}

溢出：

- 正上溢：数据大于最大正数
- 负上溢：数据小于最小负数

示例（ $n=7$ ，8位）：

- 整数补码范围：-128~+127

- 数据=130 → 正上溢；数据=-130 → 负上溢

图片信息：PPT中有图2.5，展示了定点数在数轴上的表示范围和溢出区域。

2.2.16 精度溢出

- **精度溢出：**对于小数，所有不在数轴上的小数都超出了定点小数所能表示的精度
- 只能采用舍入方法近似表示

示例（ $n=3$ ，4位，小数补码范围 $-1 \sim +7/8$ ）：

- 数轴刻度间隔 $=2^{-3}=1/8$
- 数轴上的数值： $-1, -7/8, -6/8, -5/8, -4/8, -3/8, -2/8, -1/8, 0, 1/8, 2/8, 3/8, 4/8, 5/8, 6/8, 7/8$
- $x=5/16=2.5/8 \rightarrow$ 在数轴上无法表示（精度溢出）
- 舍入近似： $x \approx 0.010=2/8=4/16$ 或 $x \approx 0.011=3/8=6/16$

图片信息：PPT中有图2.5（第20页），展示了精度溢出的概念。

2.2.17 浮点数的表示形式

核心区别：

- 定点数的小数点位置固定
- 浮点数的小数点位置不固定（可以浮动）

十进制示例： $123.456 = 0.123456 \times 10^3 = 1.23456 \times 10^2 = 12.3456 \times 10^1$

二进制浮点数表示：

$$N = 2^E \times M = 2^{\pm e} \times (\pm 0.m)$$

部分	名称	说明
E (e)	阶码 (Exponent)	纯整数，e为阶码的数值部分
M (m)	尾数 (Mantissa)	纯小数，m为尾数的数值部分

数据格式：

e_0	$e_1 e_2 \cdots e_k$	m_0	$m_1 m_2 \cdots m_n$
阶符	阶值	数符	尾数

图片信息：PPT中有图2.6（图片6.jpg），展示了浮点数的数据格式。

2.2.18 浮点数的表示范围

概念	定义
最大正数	阶码最大、尾数最大正数
最小正数	阶码最小、尾数最小正数
最大负数	阶码最小、尾数最大负数
最小负数	阶码最大、尾数最小负数
正上溢 ($+\infty$)	浮点数大于最大正数
负上溢 ($-\infty$)	浮点数小于最小负数
正下溢	正数小于最小正数（作为机器0处理）
负下溢	负数大于最大负数（作为机器0处理）
精度溢出	浮点数在表示区间但不在数轴刻度上，只能用近似数表示

重要特性：浮点数的数轴刻度是**不均匀的**，越往数轴左右两端，刻度越稀疏（与定点数的均匀刻度不同）。

图片信息：PPT中有图2.7（图片4.jpg），展示了浮点数的表示范围。

2.2.19 浮点数表示范围示例

16位浮点数（阶码5位含1位阶符用移码、尾数11位含1位数符用补码）：

- 阶码范围：-16~15
- 尾数范围：-1~ $+(1-2^{-10})$

类型	值	二进制表示
最大正数	$2^{15} \times (1-2^{-10})$	1,1111 0.11 1111 1111
最小正数	$2^{-16} \times 2^{-10}$	0,0000 0.00 0000 0001
最大负数	$2^{-16} \times (-2^{-10})$	0,0000 1.11 1111 1111
最小负数	$2^{15} \times (-1)$	1,1111 1.00 0000 0000

2.2.20 浮点数的规格化

十进制规格化：尾数为纯小数，绝对值 ≥ 0.1 且 <1

- 规格化： $123.456 = 0.123456 \times 10^3$
- 非规格化： 0.0123456×10^4 或 1.23456×10^2

二进制规格化：尾数为纯小数，绝对值 ≥ 0.5 且 <1 （最高有效位为1，原码表示）

- 规格化： $1101.1001 = 0.11011001 \times 2^{100}$
- 非规格化： $0.011011001 \times 2^{101}$ 或 1.1011001×2^{011}

2.2.21 左规与右规

左规：尾数绝对值 <0.5 ，尾数左移，阶码减1

示例：

```
1101.1001 = 0.0011011001 × 2110（非规格化）
左移1次 → 0.011011001，阶码101
左移1次 → 0.11011001，阶码100（规格化）
最终结果：0.11011001 × 2100
```

右规：尾数绝对值 ≥ 1 ，尾数右移，阶码加1

示例：

```
1101.1001 = 11.011001 × 210（非规格化）
右移1次 → 1.1011001，阶码11
右移1次 → 0.11011001，阶码100（规格化）
最终结果：0.11011001 × 2100
```

另一种规格化形式：

$$N = 2^E \times M = 2^{\pm e} \times (\pm 1.m), \quad 1 \leq |M| < 2$$

隐藏位：规格化尾数最高有效位一定是1（原码），可将该位隐藏，节省1位存储空间。

2.2.22 IEEE754浮点数标准

- 发布时间：**1985年，由IEEE发布
- 主要设计者：**威廉·卡亨（William Kahan），因此贡献获1989年图灵奖

两种主要格式：

格式	位数	数符	阶码	尾数	C语言对应
单精度	32位	1位	8位	23位	float
双精度	64位	1位	11位	52位	double

其他格式（表2.5）：半精度（16位）、扩展单精度（ ≥ 43 位）、扩展双精度（ ≥ 79 位）、四精度（128位）、八精度（256位）、Decimal32/64/128等

图片信息： PPT中有威廉·卡亨的照片（Picture2.jpg）和IEEE754数据格式图示（图片3.jpg）。

2.2.23 图灵奖简介

- **全称：**ACM A.M Turing Award
- **设立机构：**美国计算机协会（ACM），1966年设立
- **命名来源：**艾伦·麦席森·图灵（Alan M. Turing）
- **地位：**计算机领域的国际最高奖项，被誉为"计算机界的诺贝尔奖"
- **中国获奖者：**2000年，**姚期智**获图灵奖（中国人第一次也是唯一一次）
- **获奖人数最多的高校：**斯坦福大学（29位）、MIT（26位）、加州大学伯克利分校（25位）

艾伦·图灵：英国数学家、逻辑学家，被称为"计算机之父""人工智能之父"。提出图灵机模型和图灵试验。

图片信息： PPT中有艾伦·图灵的照片（Picture2.jpg）。

2.2.24 IEEE754单精度浮点数详解

阶码采用移码表示，偏移量为127（非标准移码的128）

参数	值
阶码真值范围	$E=0\sim 255$ ，真值 $=-127\sim +128$
标准移码真值	$E=0\sim 255$ ，真值 $=-128\sim +127$

尾数形式：1.M（1隐藏，只需保存M，节省1位）

符号位：S=0正数，S=1负数

真值计算公式：

情况	条件	真值
NaN	$E=255, M \neq 0$	非数（运算异常，如0除0）
$\pm\infty$	$E=255, M=0$	正无穷/负无穷
规格化数	$E=1 \sim 254$	$(-1)^S \times 2^{(E-127)} \times 1.M$
非规格化数	$E=0, M \neq 0$	$(-1)^S \times 2^{(-126)} \times 0.M$
± 0	$E=0, M=0$	$+0 / -0$

图片信息： PPT中有图2.8（图片3.jpg）和表2.6（图片4.jpg），展示了IEEE754单精度浮点数的格式和规范。

2.2.25 IEEE754浮点数表示范围

单精度：

类型	计算	近似值
规格化最小绝对值	$2^{(1-127)} \times 1.0$	$2^{(-126)}$
规格化最大绝对值	$E=254, M=11 \cdots 11$	$\approx +3.4 \times 10^{38}$
非规格化最小绝对值	$E=0, M=00 \cdots 01$	$2^{(-149)}$
非规格化最大绝对值	$E=0, M=11 \cdots 11$	$2^{(-126)} - 2^{(-149)}$

双精度：

类型	计算	近似值
规格化最小绝对值	$2^{(1-1023)} \times 1.0$	$2^{(-1022)}$
规格化最大绝对值	$E=2046, M=11 \cdots 11$	$\approx +1.8 \times 10^{308}$
非规格化最小绝对值	$E=0, M=00 \cdots 01$	$2^{(-1074)}$
非规格化最大绝对值	$E=0, M=11 \cdots 11$	$2^{(-1022)} - 2^{(-1074)}$

双精度阶码偏移量为1023（非标准移码的1024）

图片信息： PPT中有表2.7（图片9.jpg）和双精度浮点数图示（图片3.jpg）。

2.2.26 单精度浮点数与真值的转换

图片信息：PPT中有图2.10（图片3.jpg），展示了转换流程图。

例2.7：十进制20.59375 → IEEE754单精度十六进制机器码

```
20.59375 = 10100.10011（二进制）
          = 1.010010011×24（规格化）
S=0（正数），e=4，E=e+127=131=10000011
M=010010011
单精度浮点数：0 10000011 010010011000000000000000
最终机器码：0100 0001 1010 0100 1100 0000 0000 0000 = 41A4C000H
```

例2.8：IEEE754单精度 C1360000H → 十进制值

```
C1360000H = 1100 0001 0011 0110 0000 0000 0000 0000
S=1（负数），E=10000010=130，M=011011
e=E-127=130-127=3
尾数=1.M=1.011011
十进制值 = -23×1.011011 = -1011.011 = -11.375
```

2.2.27 十进制与二进制转换方法

整数部分转换（除2取余）：

```
20 = 16+4 = 10000 + 0100 = 10100
```

小数部分转换（乘2取整）：

```
0.59375×2=1.1875 → 取1
0.1875×2=0.375   → 取0
0.375×2=0.75     → 取0
0.75×2=1.5       → 取1
0.5×2=1          → 取1
0.59375 = 0.10011
```

合起来：20.59375 = 10100.10011

二进制转十进制：

$$-1011.011B = -(8+2+1 + 0.25+0.125) = -11.375$$

2.2.28 十进制整数编码 — BCD码

BCD码 (Binary Coded Decimal): 用4位二进制数表示1位十进制数

8421码: 4位二进制的权值分别为8、4、2、1

0000(0), 0001(1), 0010(2), 0011(3), 0100(4)
0101(5), 0110(6), 0111(7), 1000(8), 1001(9)

示例: 23 → 0010 0011 (一目了然); 二进制表示则为0001 0111

2421码: 4位二进制的权值分别为2、4、2、1

0000(0), 0001(1), 0010(2), 0011(3), 0100(4)
1011(5), 1100(6), 1101(7), 1110(8), 1111(9)

特性: 具有**自补性**——各位取反后正好是该数对9的补码

- 0000(0)取反→1111(9); 0001(1)取反→1110(8)

余3码: 8421码+3

0011(0), 0100(1), 0101(2), 0110(3), 0111(4)
1000(5), 1001(6), 1010(7), 1011(8), 1100(9)

BCD编码效率: 10/16 (4位二进制可表示16种状态, 只使用10种)

2.2.29 BID码与DPD码

BID码 (Binary Integer Decimal): 直接用二进制整数编码表示十进制整数

- 示例: 十进制20 → BID码=10100

DPD码 (Densely Packed Decimal): 利用10位二进制数表示3位十进制数

DPD码8种编码格式 (根据3位十进制数的大小范围):

情况	范围	示例
3个小数	(0~7)(0~7)(0~7)	375
两小一大	(0~7)(0~7)(8~9)	378
两小一大	(0~7)(8~9)(0~7)	385
两小一大	(8~9)(0~7)(0~7)	875
两大一小	(8~9)(8~9)(0~7)	895
两大一小	(8~9)(0~7)(8~9)	879
两大一小	(0~7)(8~9)(8~9)	389
3个大数	(8~9)(8~9)(8~9)	899

示例：十进制375

- $D_2=3=0abc=0011$, $D_1=7=0def=0111$, $D_0=5=0ghi=0101$
- DPD码=abcdef0ghi=0111110101

DPD编码效率：1000/1024（10位二进制表示1000种有效状态）

2.2.30 十进制浮点数

问题：二进制浮点数不能精确表示十进制数（精度溢出）

- $0.7 = 0.101100110011001100\cdots$ （无限循环）
- $1.05 = 1.0000110011001100\cdots$ （无限循环）

财务结算示例：

- $0.7 \times 1.05 = 0.735$ 元，四舍五入 $\rightarrow 0.74$ 元
- 双精度浮点计算： $0.7 \times 1.05 = 0.7349999999999999 \rightarrow 0.73$ 元（差1分钱!）

Java中的BigDecimal：采用软件方法实现十进制运算

图片信息：PPT中有十进制浮点数格式图示（图片3.jpg）。

2.2.31 IEEE754-2008的十进制浮点数格式

字段	说明
数符s	1位

字段	说明
5位组合字段comb	包含阶码和尾数的部分数据位

三种格式：

格式	条件	comb内容
格式①	尾数最高位=0~7	高2位为阶码最高有效位，低3位为尾数最高有效位
格式②	尾数最高位=8~9	高2位为11，后续2位为阶码最高有效位，最低位区分8/9
格式③	无穷大或非数	高4位为1111

计算公式： $N = (-1)^s \times 10^{(E-bias)} \times T$

格式	位数	bias	E的范围
_Decimal32	s(1)+comb(5)+E(6)+T(20)	101	-101~90
_Decimal64	s(1)+comb(5)+E(8)+T(50)	398	-398~369
_Decimal128	s(1)+comb(5)+E(12)+T(110)	6176	-6176~6111

注意：基数为10，尾数T是定点整数（非定点小数）

2.2.32 汇编语言中的数据类型

- 操作数是定点数还是浮点数、有符号还是无符号，完全取决于指令操作符

不同指令集的数据运算类型（表2.11）：

标记	含义
U	无符号数
I	有符号数
F	浮点数
S	单精度浮点数
D	双精度浮点数

图片信息：PPT中有表2.11（图片3.jpg），展示了不同指令集的数据运算类型。

2.2.33 C语言中的数据类型

整数类型：

类型	位数	有符号范围（补码）	无符号范围
char	8位	-128~+127	0~255
short	16位	-32768~+32767	0~65535
int	32位	$-2^{31} \sim +2^{31}-1$	$0 \sim 2^{32}-1$
long	64位	$-2^{63} \sim +2^{63}-1$	$0 \sim 2^{64}-1$

浮点数类型：

类型	位数	说明
float	32位	单精度浮点数
double	64位	双精度浮点数
_Float16	16位	半精度浮点数
long double	128位	四精度浮点数
_Decimal32/64/128	32/64/128位	十进制浮点数

默认为有符号数，加"unsigned"表示无符号数。

2.2.34 C语言整型数据溢出示例

4位无符号数（范围0~15）：

- $7+9=16 > 15 \rightarrow$ 无符号溢出
- $7-9=-2 < 0 \rightarrow$ 无符号溢出

4位有符号数补码（范围-8~+7）：

- $3+5=8 > +7 \rightarrow$ 正上溢
- $3-(-5)=8 > +7 \rightarrow$ 正上溢
- $-5+(-6)=-11 < -8 \rightarrow$ 负上溢
- $-5-6=-11 < -8 \rightarrow$ 负上溢

图片信息： PPT中有图2.12(a)和(b)（图片4.jpg和图片13.jpg），展示了无符号数和有符号数的溢出情况。

2.2.35 C语言数据表示实例

同一机器码C77FFFFFF在不同数据类型下的解释：

数据类型	机器码	十进制值
32位整数（补码）	1100 0111 0111...	-947,912,705
无符号整数	1100 0111 0111...	3,347,054,591
IEEE754浮点数	S=1,E=10001110,M=111...	-65535.99609375

图片信息： PPT中有该实例的代码截图（图片5.jpg、图片3.jpg、图片2.jpg）。

2.2.36 C语言运算溢出实例

4种运算溢出：

运算	结果	说明
32767+1	-32768	短整数溢出，应改用32位整数
-32768-3	32765	短整数溢出
128+255	127	无符号字符溢出，应改用16/32位
128-255	129	无符号字符溢出

溢出原理：

$32767+1 = 0111\ 1111\ 1111\ 1111 + 1 = 1000\ 0000\ 0000\ 0000 = -32768$ （补码）
 $128+255 = 1000\ 0000 + 1111\ 1111 = 0111\ 1111 = 127$ （无符号数）

图片信息： PPT中有代码截图（图片8.jpg、图片12.jpg）展示溢出实例。

2.2.37 C语言整型数据类型转换

相同字长转换（8位转8位）：

转换	结果	说明
FF(无符号) → 有符号字符	-1	FF=255（无符号），FF=1111 1111=-1（补码）
-127(有符号) → 无符号字符	129	-127=81H=1000 0001，无符号解释=129

小字长转大字长（8位转32位）：

转换	结果	说明
254(无符号) → 32位整数	0000 00FE	零扩展
254(有符号) → 32位整数	FFFF FFFE	符号扩展（254=-2）
FE(有符号) → 32位无符号	FFFF FFFE = 4294967294	先符号扩展再解释

大字长转小字长（32位转16位）：

转换	结果	说明
0xFFFF1001 → 短整数	0x1001 = 4097	截断高16位
0xFFFF1001 → 无符号短整数	0x1001 = 4097	截断高16位

图片信息：PPT中有代码截图和输出结果（图片4.jpg、图片13.jpg、图片3.jpg、图片14.jpg、图片5.jpg、图片10.jpg）。

2.2.38 C语言浮点数据类型转换

转换方向	是否安全	可能问题
float → double	安全	无问题
double → float	不安全	溢出（指数过大）或精度丢失（舍入）
float/double → int	不安全	溢出或精度丢失（舍入）
int → float	不安全	精度溢出（int数轴均匀，float数轴不均匀）
int → double	安全	无问题

关键表达式恒成立分析（表2.13）：

表达式	是否恒成立	原因
i==(int)(float)i	否	int转float精度溢出（float尾数位不足23位）
i==(int)(double)i	是	int转double无问题
f==(float)(int)f	否	float转int溢出或精度丢失
f==(float)(double)f	是	float转double无问题
d==(float)d	否	double转float溢出或精度丢失
f==-(f)	是	浮点数原码表示，单目运算仅符号取反

表达式	是否恒成立	原因
$(d+f)-d==f$	否	浮点数不满足结合律 (d大f小, 左边结果可能为0)

2.2.39 例2.9 (综合题)

题目: f1使用int型, f2使用float型, 计算f(n)。

```
int f1(unsigned n) {
    int sum=1, power=1;
    for(unsigned i=0; i<=n-1; i++) {
        power *= 2;
        sum += power;
    }
    return sum;
}

float f2(unsigned n) {
    float sum=1, power=1;
    for(unsigned i=0; i<=n-1; i++) {
        power *= 2;
        sum += power;
    }
    return sum;
}
```

五个子问题的答案:

(1) $n=0$ 时f1死循环的原因:

- i和n都是unsigned (32位), $n=0$ 时 $n-1=0-1=11\cdots11$ (32个1) $=2^{32}-1$
- " $i<=n-1$ "永远成立 $\rightarrow$ 死循环
- 若改为int型, $n=0$ 时 $n-1=-1$, 条件不成立, 不会死循环

(2) f1(23)和f2(23)返回值相等:

- $f1(23)=00FF\ FFFFH=16,777,215$
- $f2(23)=4B7F\ FFFFH=16,777,215$

(3) $f1(24)\neq f2(24)$ 的原因:

- $f1(24)=33,554,431$; $f2(24)=33,554,432.0$

- float尾数只有23位，f(24)的尾数有24位，舍入处理末位加1

(4) $f(31)=2^{32}-1$ ， $f1(31)=-1$ 的原因：

- f1的返回值sum是int型，FFFF FFFFH=-1（补码）
- 要使 $f1(n)=f(n)$ ，最大 $n=30$

(5) $f2(127)=7F80\ 0000H=+\infty$ ：

- 要使 $f2(n)$ 不溢出，最大 $n=126$

2.3 非数值数据的表示

2.3.1 ASCII码

- ASCII (American Standard Code for Information Interchange)：美国信息交换标准代码
- 用7位二进制数表示128个字符
- 扩展ASCII码：8位表示256个字符，前128个与标准ASCII相同

相关术语：

- MSB (Most Significant Bit)：最高有效位
- LSB (Least Significant Bit)：最低有效位

图片信息：PPT中有表2.14标准ASCII码 (Picture2.jpg) 和扩展ASCII码 (Picture2.jpg)。

2.3.2 汉字编码

国标码 (GB2312)：

- 2个字节（16位）表示1个汉字
- 每个字节的MSB均为1，实际上1个汉字用14位表示
- 包含7445个字符：6763个常用汉字 + 682个全角非汉字字符

区位码：

- 94行（区）×94列（位）
- 区位码 + A0A0H = 国标码

GBK标准：

- GB表示国标、K表示扩展
- 不要求每个字节MSB必须为1
- 增加了20,000个新汉字和符号

GB18030标准：

- 4字节汉字编码
- 支持少数民族文字

UTF编码：UTF-8、UTF-16、UTF-32

Unicode编码：统一码

Big5（大五码）：

- 繁体中文社区最常用
- 共收录13,060个汉字

2.3.3 汉字处理流程

编码类型	说明
汉字输入码（外码）	使用英文键盘输入汉字时的编码
汉字机内码	计算机内部存储、处理、传输汉字时所用的统一编码

汉字输入码分类：

类型	示例
流水码	国标码、区位码
音码	拼音码（全拼、简拼、双拼）
形码	五笔字型码
音形码	自然码、钱码

汉字字形码：

- 汉字的输出码（字型码）
- 字形点阵：16×16、24×24、32×32、48×48
- 1个32×32点阵汉字需要4B×32=128B存储空间
- 汉字库：存放汉字字形码的字库

2.4 数据信息的校验

2.4.1 校验码的基本概念

- **校验码**：具有发现错误或纠正错误能力的数据编码
- **组成**：校验码(k+r位) = 原始数据(k位) + 校验数据(r位)
- 校验数据按照某种规则通过编码电路编码
- 出现错误时会破坏预定规则，通过解码电路可以发现或纠正错误

图片信息：PPT中有校验码的编码和解码示意图（图片4.jpg、图片6.jpg）。

2.4.2 码距

- **码距（海明距离）**：两个编码对应二进制位不同的个数
- 示例：10101和00110，第1、4、5位不同，码距为3
- **编码集的码距**：任意两个码字的最小码距

码距与检错/纠错能力的关系：

条件	能力
$d \geq e+1$	可检测e个错误
$d \geq 2t+1$	可纠正t个错误
$d \geq e+t+1$ 且 $e \geq t$	可检测e个错误并纠正t个错误

具体对应表：

码距d	检错e	纠错t	检错e且纠错t
1	0	0	0,0
2	1	0	1,0
3	2	1	1,1
4	3	1	2,1
5	4	2	2,1
6	5	2	3,2
7	6	3	3,3

核心规律：码距越大，抗干扰能力、纠错能力越强；但数据冗余越大，编码效率越低。

2.4.3 例2.10 — 码距分析

(1) 4位二进制表示16种状态 (0000~1111):

- 码距为1 (如0000和0001只有1位不同)
- 任何合法编码发生1位错会变成另一个合法编码
- 不具备检测错误的能力

(2) 4位二进制表示8种状态 (0000、0011、0101、0110、1001、1010、1100、1111):

- 最小码距为2
- 任何合法编码发生1位错会变成无效编码 → 可以识别1位错误
- 发生2位错会变成另一个合法编码 → 对2位错误无法检测

2.4.4 简单奇偶校验

奇校验： 校验码中1的个数为奇数个

原始数据	校验位	校验码
000 0000	1	0000 0001
111 1111	0	1111 1110
101 1001	1	1011 0011

偶校验： 校验码中1的个数为偶数个

原始数据	校验位	校验码
000 0000	0	0000 0000
111 1111	1	1111 1111
101 1001	0	1011 0010

公式表示：

- 奇校验位： $P = \text{非}(D_1 \oplus D_2 \oplus \cdots \oplus D_n)$
- 偶校验位： $P = D_1 \oplus D_2 \oplus \cdots \oplus D_n$
- 奇校验检错位： $G = \text{非}(D'_1 \oplus D'_2 \oplus \cdots \oplus D'_n \oplus P')$
- 偶校验检错位： $G = D'_1 \oplus D'_2 \oplus \cdots \oplus D'_n \oplus P'$

判断：

- $G=0$: 无奇数位错 (无错, 或2位错, 或4位错...)
- $G=1$: 有奇数位错 (1位错, 或3位错, 或5位错...)

局限: 简单奇偶校验只能发现1位错, 但不能纠正错误。

2.4.5 交叉奇偶校验

问题: 简单奇偶校验只有1位校验位, 只能发现1位错, 不能纠错。

解决方案: 多重奇偶校验——每个数据位至少位于两个校验组中。

示例: 4个原始数据R3、R2、R1、R0, 每个7位

- $R3=1010110$ 、 $R2=1110110$ 、 $R1=0010001$ 、 $R0=1100100$
- 每行产生1个偶校验位Pr (4个)
- 每列产生1个偶校验位Pc (7个)
- 1个公共偶校验位
- 共12个校验位

交叉偶校验表:

	C6	C5	C4	C3	C2	C1	C0	Pr
R3	1	0	1	0	1	1	0	0
R2	1	1	1	0	1	1	0	1
R1	0	0	1	0	0	0	1	0
R0	1	1	0	0	1	0	0	1
Pc	1	0	1	0	1	0	1	0

1位错纠错: 假设R3最高位出错→根据行检错码Gr和列检错码Gc定位到具体位, 取反即可纠正。

2位错发现: 假设R3最高位和次高位出错→Gc可发现2位错, 但Gr无1, 无法定位具体行。

结论: 交叉奇偶校验可以发现并纠正1位错, 可以发现2位错但不能纠正。

2.4.6 海明校验 — 基本概念

- **提出者:** 理查德·海明 (Richard Hamming), 1950年
- **本质:** 多重奇偶校验码 (Error-Correcting Codes, ECC码)

- SEC码 (Single-bit Error Correction): 能纠正1位错, 最小码距为3

校验位位数:

- 原始数据k位, 校验码r位, 海明码n位 ($n=k+r$)
- 满足关系: $n = k+r \leq 2^r-1$

k值	r值	验证
1	2	$3 \leq 2^2-1=3 \checkmark$
2~4	3	$5 \sim 7 \leq 7 \checkmark$
5~11	4	$9 \sim 15 \leq 15 \checkmark$

编码效率 = $k/(k+r)$; $k=8$ 、 $r=4$ 时, 效率= $8/12=66.7\%$

图片信息: PPT中有理查德·海明的照片 (Picture2.jpg)。

2.4.7 海明校验 — 编码分组规则

问题1: 校验码位于海明码的什么位置?

- 答: 第1、2、4、8、16、...位 (从右往左)

问题2: 如何得到校验码的值?

- 以(7,4)码为例 ($k=4$, $r=3$, $n=7$):
 - $P_1 = D_1 \oplus D_2 \oplus D_4$
 - $P_2 = D_1 \oplus D_3 \oplus D_4$
 - $P_3 = D_2 \oplus D_3 \oplus D_4$

问题3: 如何得到检错码的值?

- $G_1 = P'_1 \oplus D'_1 \oplus D'_2 \oplus D'_4$
- $G_2 = P'_2 \oplus D'_1 \oplus D'_3 \oplus D'_4$
- $G_3 = P'_3 \oplus D'_2 \oplus D'_3 \oplus D'_4$

2.4.8 海明校验 — 检错与纠错

(7,4)码检错码与出错位对应关系:

$G_3G_2G_1$	状态
000	无错
001	H_1 出错
010	H_2 出错
011	H_3 出错
100	H_4 出错
101	H_5 出错
110	H_6 出错
111	H_7 出错

补充例1：原始数据=1010，求海明码并验证纠错

- $P_1=0\oplus 1\oplus 1=0$, $P_2=0\oplus 0\oplus 1=1$, $P_3=1\oplus 0\oplus 1=0$
- 海明码=1010010
- 假设 H_7 出错 $\rightarrow$ 0010010
- $G_1=1$, $G_2=1$, $G_3=1 \rightarrow G_3G_2G_1=111 \rightarrow H_7$ 出错，取反纠正

补充例2：接收海明码=1010110，求原始数据

- $G_1=1$, $G_2=1$, $G_3=0 \rightarrow G_3G_2G_1=011 \rightarrow H_3$ 出错
- 正确海明码=1010010 $\rightarrow$ 原始数据=1010

2.4.9 扩展海明码（SECDED码）

- SECDED (Single Error Correction Double Error Detection)
- 最小码距为4
- 可以同时检测2位错并纠正1位错

实现方法：在海明码基础上增加一个总偶校验码 P_{all}

$$P_{all} = (D_1 \oplus D_2 \oplus \dots \oplus D_k) \oplus (P_1 \oplus P_2 \oplus \dots \oplus P_r)$$

检错逻辑：

G_{all}	$G=G_r \cdots G_1$	结论
0	0	无错
1	0	1位错, P_{all} 出错, 不需纠错
1	$\neq 0$	1位错, 根据 G 的值纠错

Gall	$G=G_r \cdots G_1$	结论
0	$\neq 0$	2位错, 无法纠错

实际应用：服务器ECC校验内存

- 64位数据宽度，7位海明校验位+1位总校验位=72位
- 16GB ECC内存实际容量=18GB (16GB×72/64)

2.4.10 例2.11 — 7位ASCII码的海明码

原始数据= $D_7 \cdots D_1=1101010$, $k=7$, $r=4$

校验码：

- $P_1 = D_1 \oplus D_2 \oplus D_4 \oplus D_5 \oplus D_7 = 0 \oplus 1 \oplus 1 \oplus 0 \oplus 1 = 1$
- $P_2 = D_1 \oplus D_3 \oplus D_4 \oplus D_6 \oplus D_7 = 0 \oplus 0 \oplus 1 \oplus 1 \oplus 1 = 1$
- $P_3 = D_2 \oplus D_3 \oplus D_4 = 1 \oplus 0 \oplus 1 = 0$
- $P_4 = D_5 \oplus D_6 \oplus D_7 = 0 \oplus 1 \oplus 1 = 0$

海明码=11001010011

该海明码不能区分1位错和2位错（需要使用扩展海明码）。

2.4.11 循环冗余校验（CRC）— 模2运算

模2运算规则：

- 加减法： $0 \pm 0 = 0$, $0 \pm 1 = 1$, $1 \pm 0 = 1$, $1 \pm 1 = 0$ （等价于异或）
- 乘法：根据模2加法求部分积，不考虑进位
- 除法：根据模2减法求部分余数，首位为1商1，首位为0商0

示例：

- $1101 \times 101 = 111001$
- $10010 \div 101 = \text{商}101, \text{余数}11$

图片信息：PPT中有模2运算的图示（图片16.jpg、图片21.jpg、图片23.jpg）。

2.4.12 CRC编码规则

原始数据= $C_{k-1}\cdots C_1C_0$ (k位), 校验码= $P_{r-1}\cdots P_1P_0$ (r位)

CRC码= $C_{k-1}\cdots C_1C_0P_{r-1}\cdots P_1P_0$ ($n=k+r$ 位), 称为(n,k)码

满足关系: $n=k+r \leq 2^r-1$ (与海明码相同)

编码步骤:

1. 原始数据用多项式 $M(x)$ 表示
2. $M(x)$ 左移 r 位: $M(x) \cdot 2^r$
3. 选择 $r+1$ 位生成多项式 $G(x)$
4. $M(x) \cdot 2^r$ 按模2除以 $G(x)$, 得商 $Q(x)$ 和余数 $R(x)$
5. CRC码 = $M(x) \cdot 2^r + R(x) = Q(x) \cdot G(x)$

关键性质: CRC码一定能被生成多项式整除

示例: 原始数据=110, $k=3$, $r=4$, $n=7$

- $M(x) \cdot 2^r = 110\ 0000$
- $G(x) = 11101$
- $1100000 \div 11101 = \text{商}101、\text{余数}1001$
- CRC码=110 1001

2.4.13 CRC编码特性

CRC(7,3)码可以发现1位错并纠正1位错 (每个出错位对应不同余数):

余数	出错位
0000	无错
0001	第1位
0010	第2位
0100	第3位
1000	第4位
1101	第5位
0111	第6位
1110	第7位

2位错可以发现但不能纠正 (每个余数对应3个不同的2位错)。

2.4.14 CRC编解码电路与流程

编码：原始数据从Din端串行输入，经过n-1个时钟周期后得到余数，原始数据和余数拼接为CRC码。

解码：接收到的CRC码从Din端串行输入，经过n-1个时钟周期后得到余数；根据余数值判断有无错及哪一位出错。

图片信息：PPT中有图2.20 CRC串行编解码电路、图2.21 CRC编解码流程图、以及电路操作步骤截图（图片20.jpg、图片18.jpg、图片16.jpg、图片6.jpg、图片3.jpg、图片1.jpg、图片4.jpg）。

2.4.15 CRC生成多项式

生成多项式的要求：

1. 最高位和最低位必须为1
2. CRC码任何1位错，被G(x)模2除后余数不为0
3. 不同位错误，余数不同
4. 对余数继续模2除，应使余数循环

常用生成多项式：

- $G(x)=x+1$
- $G(x)=x^3+x+1$
- $G(x)=x^4+x+1$
- $G(x)=x^5+x^4+x^2+1$
- $G(x)=x^5+x^3+1$

2.4.16 CRC检错性能

采用r位校验码的CRC具有如下检错能力：

错误类型	检错能力
突发长度 $\leq r$	检测出所有
突发长度 $=r+1$	检测出 $(1-2^{-(r-1)})$ 比例
突发长度 $>r+1$	检测出 $(1-2^{-r})$ 比例
小于最小码距的任意位数错	检测出所有
奇数位错（G(x)中1的个数为偶数时）	检测出所有

r=16时的性能:

- 检测出所有突发长度 ≤ 16 的错误
- 检测出 $\approx 99.997\%$ 突发长度=17的错误
- 检测出 $\approx 99.998\%$ 突发长度 > 17 的错误

CRC应用:

- X.25 FCS: CRC-CCITT
- WinRAR、ARJ、LHA: CRC32
- 磁盘驱动器: CRC16
- GIF、TIFF图像格式: CRC

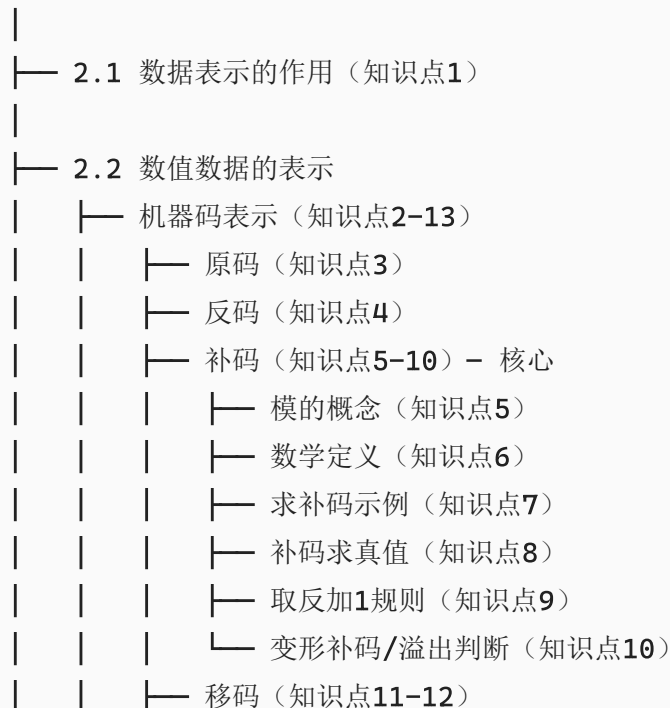
2.4.17 实践训练 (实验1)

1. 在Logisim中设计包含**16位数据位**的海明码编解码电路, 要求能够在假设没有3位错的前提下检测出两位错并纠正一位错。
2. 利用组合逻辑电路在Logisim中设计一个包含**16位数据位**的并行CRC编解码电路, 要求能够在假设没有3位错的前提下检测出两位错并纠正一位错。

知识点关联图谱

整体结构关联

第2章 数据信息的表示



- | | | └─ 4种机器码对比（知识点13）
- | | └─ 定点数表示（知识点14-17）
 - | | | └─ 定点小数/整数（知识点14-15）
 - | | | └─ 表示范围（知识点16）
 - | | | └─ 精度溢出（知识点17）
- | | └─ 浮点数表示（知识点18-28）
 - | | | └─ 表示形式（知识点18）
 - | | | └─ 表示范围（知识点19-20）
 - | | | └─ 规格化/左规右规（知识点21-22）
 - | | | └─ IEEE754标准（知识点23-26）
 - | | | └─ 转换流程（知识点27-28）
 - | | | └─ 图灵奖（知识点24）
- | | └─ 十进制编码（知识点29-32）
 - | | | └─ BCD码（知识点29）
 - | | | └─ BID/DPD码（知识点30）
 - | | | └─ 十进制浮点数（知识点31-32）
- | | └─ 数据类型（知识点33-40）
 - | | | └─ 汇编/C语言类型（知识点33-34）
 - | | | └─ 溢出实例（知识点35-37）
 - | | | └─ 类型转换（知识点38-39）
 - | | | └─ 综合例题（知识点40）
- | |
- | └─ 2.3 非数值数据的表示（知识点41-43）
 - | | └─ ASCII码（知识点41）
 - | | └─ 汉字编码（知识点42-43）
- | |
- | └─ 2.4 数据信息的校验（知识点44-60）
 - | | └─ 码距概念（知识点45-46）
 - | | └─ 奇偶校验（知识点47-48）
 - | | └─ 海明校验（知识点49-53）
 - | | | └─ 基本概念（知识点49）
 - | | | └─ 编码规则（知识点50）
 - | | | └─ 检错纠错（知识点51）
 - | | | └─ 扩展海明码（知识点52-53）
 - | | └─ 循环冗余校验CRC（知识点54-59）
 - | | | └─ 模2运算（知识点54）
 - | | | └─ 编码规则（知识点55）
 - | | | └─ 编码特性（知识点56）
 - | | | └─ 编解码电路（知识点57）
 - | | | └─ 生成多项式（知识点58）
 - | | | └─ 检错性能（知识点59）

核心关联关系

关联关系	说明
原码/反码 → 补码	原码和反码都有正负零问题，补码解决此问题成为主流
补码 → 变形补码	变形补码在补码基础上增加符号位用于溢出判断
补码 → 移码	移码=补码符号位取反，用于浮点数阶码比较
定点数 → 浮点数	浮点数通过阶码和尾数扩展了表示范围
规格化 → 隐藏位	规格化尾数最高位必为1，可隐藏节省1位
浮点数 → IEEE754	IEEE754是浮点数的工业标准，规定了格式和运算规则
数据类型 → 溢出	不同数据类型有不同表示范围，超出则溢出
校验码 → 码距	码距决定检错纠错能力，是校验码设计的理论基础
海明校验 → 扩展海明码	扩展海明码增加总校验位，实现检2纠1
CRC → 模2运算	CRC基于模2运算（异或），可用移位寄存器硬件实现

文档整理完毕，共60个知识点